

# e2m2e 技术白皮书

地月空间任务规划算法工具集：架构、算法与验证

e2m2e 项目团队

<https://github.com/cislunarspace/e2m2e>

v5.8.7 2026 年 8 月

## 摘要

e2m2e (Earth to Moon, Moon to Earth) 是面向地月空间任务规划的开源算法工具集。在“LLM + Agent”式自主任务规划系统中，大语言模型负责理解任务意图并分解、编排子任务，e2m2e 则提供精确可靠的轨道计算能力：构建地月空间动力学模型，生成周期轨道族，设计轨道间的转移路径，并对结果进行可视化检验。本白皮书介绍 e2m2e 的系统架构 (Python 编排 + Rust 数值内核的分层设计)、核心算法 (微分修正、多重打靶、延拓、不变流形拼接、Lambert/低能量/低推力转移)、时空基准与验证体系，以及 Facade/CLI/MCP 三种调用接口。

## 目录

|     |                        |   |
|-----|------------------------|---|
| 1   | 引言                     | 4 |
| 1.1 | 地月空间任务规划的计算需求          | 4 |
| 1.2 | 定位：LLM + Agent 时代的数值基座 | 4 |
| 1.3 | 与现有工具的关系               | 4 |
| 1.4 | 本报告的组织                 | 4 |
| 2   | 系统架构                   | 5 |
| 2.1 | 从一条 NRHO 轨道说起          | 5 |
| 2.2 | 运行时分层                  | 5 |
| 2.3 | 五个架构模块                 | 6 |
| 3   | 时空基准与动力学内核             | 6 |
| 3.1 | 时空基准：时间、坐标、常数          | 6 |
| 3.2 | 三层动力学模型                | 7 |
| 3.3 | 高保真力模型                 | 8 |

|                                         |           |
|-----------------------------------------|-----------|
| 3.4 Rust 积分器内核 . . . . .                | 8         |
| <b>4 周期轨道设计</b>                         | <b>9</b>  |
| 4.1 轨道族总览 . . . . .                     | 9         |
| 4.2 初值猜测：从解析近似出发 . . . . .              | 10        |
| 4.3 微分修正：利用对称性的 STM 牛顿迭代 . . . . .      | 10        |
| 4.4 多重打靶：长弧段的星历修正 . . . . .             | 10        |
| 4.5 延拓：从一条轨道到一个族 . . . . .              | 11        |
| 4.6 稳定性分析与不变流形 . . . . .                | 11        |
| 4.7 设计流水线 . . . . .                     | 11        |
| <b>5 转移轨道设计</b>                         | <b>12</b> |
| 5.1 脉冲转移 . . . . .                      | 12        |
| 5.1.1 Lambert 求解与 porkchop 扫描 . . . . . | 12        |
| 5.1.2 三体打靶 . . . . .                    | 12        |
| 5.1.3 多脉冲优化与 Lawden 主矢量检验 . . . . .     | 12        |
| 5.2 低能量转移 . . . . .                     | 13        |
| 5.2.1 月球借力 (LGA) . . . . .              | 13        |
| 5.2.2 WSB 弹道捕获 . . . . .                | 13        |
| 5.2.3 不变流形拼接 . . . . .                  | 13        |
| 5.3 低推力转移 . . . . .                     | 14        |
| 5.4 网格搜索 + NLP 两步优化框架 . . . . .         | 14        |
| <b>6 轨道控制与 Monte Carlo 仿真</b>           | <b>14</b> |
| 6.1 三种控制律 . . . . .                     | 15        |
| 6.2 Monte Carlo 误差仿真 . . . . .          | 15        |
| <b>7 接口、数据与生态</b>                       | <b>15</b> |
| 7.1 Facade：唯一的任务级入口 . . . . .           | 15        |
| 7.2 MCP：Agent 原生接口 . . . . .            | 16        |
| 7.3 轨道库 catalog . . . . .               | 16        |
| 7.4 安装与部署 . . . . .                     | 16        |
| <b>8 与现有工具的对比分析</b>                     | <b>17</b> |
| 8.1 对比范围与方法 . . . . .                   | 17        |
| 8.2 生态全景 . . . . .                      | 17        |

- 8.3 核心差异化轴：地月三体设计自动化 . . . . . 17
- 8.4 代表性对象逐评 . . . . . 17
  - 8.4.1 通用任务分析平台：GMAT、STK、ATK . . . . . 19
  - 8.4.2 开源天体力学库：Orekit、pykep、poliastro . . . . . 19
  - 8.4.3 自建路线与精度标尺：SciPy . . . . . 20
  - 8.4.4 生态互补件：r2s2、Cesium、ootk . . . . . 20
- 8.5 性能与精度：证据的分区陈述 . . . . . 20
- 8.6 组合生态：地月设计引擎 . . . . . 21
- 9 验证体系与结语 . . . . . 22
  - 9.1 按定义验证 . . . . . 22
  - 9.2 结语与展望 . . . . . 22

## 1 引言

### 1.1 地月空间任务规划的计算需求

随着 Artemis 计划、Gateway 空间站、嫦娥探月工程以及商业月球任务的推进，地月空间（cislunar space）正成为航天任务设计的热点区域。与近地轨道不同，地月空间的动力学由地球与月球的双引力主导：拉格朗日平动点附近的周期轨道（Halo、NRHO、DRO 等）没有解析解，其设计依赖三体问题框架下的数值方法——微分修正、多重打靶、参数延拓与不变流形理论；而轨道之间的转移设计则需要在脉冲、低能量与低推力等多种推进模式之间权衡。这类计算具有三个共同特征：**模型多**（圆型限制性三体问题 CR3BP、星历  $N$  体、双圆四体问题 BCR4BP 等多保真度模型并存）、**迭代重**（一次星历修正往往涉及上万段轨道积分）、**链条长**（初猜、修正、高保真传播、控制仿真环环相扣）。

### 1.2 定位：LLM + Agent 时代的数值基座

e2m2e（Earth to Moon, Moon to Earth）面向上述需求，定位是 **地月空间任务规划的算法工具集基座**。在“LLM + Agent”式自主任务规划系统中，大语言模型负责理解任务意图、分解与编排子任务；e2m2e 负责数值计算的那一半——构建地月空间动力学模型，生成周期轨道族，设计轨道之间的转移路径，并把结果画出来供检查。为此，e2m2e 在传统的 Python API 与 CLI 之外，原生提供 MCP（Model Context Protocol）服务：13 个任务级工具通过 stdio 暴露给 LLM Agent，工具清单由 Facade 方法元数据派生，计算产物自动归档、`record_id` 跨工具链接，使得“用自然语言设计一条 NRHO 并做 100 次 Monte Carlo 轨道保持仿真”成为一条可执行的指令。

### 1.3 与现有工具的关系

- **GMAT / STK**：通用任务分析工具，以图形界面和脚本驱动为主，验证数据充分但难以被 Agent 程序化调用；
- **Orekit / poliastro**：通用轨道力学库，覆盖近地到深空，但不以地月三体问题为中心，也无任务级封装；
- **e2m2e**：专注地月空间，把“选族—初猜—修正—高保真传播—转移—保持”整条设计链封装为任务级接口，数值内核用 Rust 实现，并以 MCP 原生对接 LLM Agent。

e2m2e 不追求替代通用工具，而是与其形成互补：传播精度对齐 GMAT 与 DFH 至亚百米量级（见第 9 节），坐标与时间链用 GMAT 参考数据交叉验证，同时提供通用工具所不具备的周期轨道族生成与 Agent 原生接口。第 8 节基于对 10 个同类/生态工具的代码级调研，给出系统的对比分析。

### 1.4 本报告的组织

第 2 节介绍系统架构：五个架构模块与“Python 编排 + Rust 数值内核”的分层运行时；第 3 节介绍时空基准、动力学模型与高保真力模型；第 4 节与第 5 节分别介绍周期轨道设计与转移轨

道设计的核心算法；第 6 节介绍轨道控制与 Monte Carlo 仿真；第 7 节介绍 Facade/CLI/MCP 接口与轨道库；第 8 节将 e2m2e 与 GMAT、STK、ATK、Orekit、pykep 等 10 个工具对比定位；第 9 节给出验证体系并总结展望。

## 2 系统架构

### 2.1 从一条 NRHO 轨道说起

架构决策很少是凭空做出的。以“设计一条地月 L2 的 NRHO 轨道”为例，一条任务请求穿过系统的每个环节，恰好暴露了每层存在的理由：

- **时间**。用户输入 2024-01-01 00:00:00 UTC，但 DE 历表的自变量是质心力学时 TDB，两者此刻相差约 69 秒。月球以约 1 km/s 绕地球公转，拿 UTC 当 TDB 查位置会错出约 70 km，近月制动窗口完全作废。因此 e2m2e 内部统一 TDB，只在接口边界转 UTC——与 STK 的做法一致。
- **坐标**。CR3BP 初猜在地月会合系给出，任务书参数在 J2000 惯性系，测控站在 ITRF93 地固系。换系不换数就是另一个物理位置，且这类错误编译器不会报错（1999 年火星气候轨道器的单位混淆是同类教训）。
- **常数**。地球 GM 有 DE440/DE421/WGS-84 多个值，地月质量比  $\mu$  亦然。e2m2e 自身曾在此出错：Python 与 Rust 各抄一个  $\mu$ ，算出的轨道族对不上文献。常量基准集（多套并存、每套内部自治、双语言同源）为此而设。
- **计算规模**。一次 2 年 50 圈的星历修正约需两万段轨道积分，每段数千步、每步评估力模型并查询星历，标量运算以十亿计。这一层必须快且可并行；而 CSPICE 内核是全局状态、不可并发——所以 Rust 层不持有 SPICE 句柄，只吃预采样注入的星历缓存表。
- **编排**。选 DRO 还是 Halo、初猜振幅多大、容差多严、预报多久，这些是天天在变的领域决策，留在 Python。

### 2.2 运行时分层

运行时架构只有四块，外加两块支撑（图 1）：

- **e2m2e/api**：唯一对外入口。任务级 Facade，CLI 与 MCP 服务均由 Facade 方法元数据同源派生；
- **e2m2e/algorithm**：用领域知识构造问题——选轨道族、定约束、给初猜（种子来自轨道库记录或 normal\_form 模块）；只做三件事：构造问题、调 Rust 迭代器、解释结果，不做数值迭代；
- **crates/**：Rust 数值层，重迭代在此收敛。六个 crate：spice（CSPICE FFI + 缓存）、propagation（纯数学积分器）、forces（力模型 + STM）、integrators（pyo3 绑定 + 打靶等迭代求解器 + 并行）、levelset（水平集/HJB 求解内核）、hjb-dynamics（HJB 的 Hamiltonian 动力学实现）；
- **e2m2e/data**：星历缓存表、坐标数据（EOP/闰秒）、常量基准集；

- e2m2e/tools: 日志与可视化支撑; e2m2e/mbse: MBSE 数据模型与图生成, 在依赖链之外。



图 1: e2m2e 运行时分层。依赖单向向下: 接口层同源派生 CLI/MCP; 编排层只构造问题不做迭代; Rust 数值层不持有 SPICE 句柄, 星历以预采样缓存表注入。

一条轨道任务的旅程: api 接收请求 → algorithm/family 选族与初猜 → 打靶下沉至 crates 积分器, 逐步受力由 forces crate 评估 (星历来自 data/frames 的预采样缓存表, 常数来自 data/constants) → 结果落入 catalog/ 轨道库, 经 CLI/MCP 交付。

### 2.3 五个架构模块

在运行时分层之上, 工程组织划分为五个架构模块 (表 1)。运行时与模块是两个视角: 前者描述计算如何流动, 后者描述工程如何维护。

#### 设计要点

数值下沉的边界由“迭代的密度”决定: 打靶、延拓、NSGA-II 演化算子、低推力直接法等重迭代内核已在 Rust 侧; Python 保留外层编排与参照路径。Rust 层不持有 SPICE 句柄这一约束, 使整个数值内核天然线程安全, 可以 Rayon 并行。各算法模块的迁移进度逐项登记在 docs/architecture/numerics-migration-status.md。

## 3 时空基准与动力学内核

### 3.1 时空基准: 时间、坐标、常数

轨道计算的误差常常不出在算法, 而出在基准。e2m2e 把时空基准作为独立模块维护:

**时间。**动力学内部统一使用质心力学时 TDB (SPICE ET 秒或 JD\_TDB), UTC 只出现在接口边界; 转换链覆盖 UTC / TAI / TT / TDB, 闰秒内核自动加载。

表 1: 五个架构模块

| 模块        | 职责                                                                                                                                           |
|-----------|----------------------------------------------------------------------------------------------------------------------------------------------|
| 时空系统与常量   | 时间尺度（UTC/TDB/TAI/TT，动力学统一 TDB）；参考系转换（J2000/ITRF93/IAU 2006、GCRS-EBCRS、动态轴）；物理常量基准集（DE421/DE440/WGS84 多套并存，Python/Rust 同源防漂移）                 |
| Rust 计算   | 六个 crate（见上）；迭代问题由 Python 构造传入，Rust 只迭代到收敛；星历以缓存表注入，可并行                                                                                      |
| Python 编排 | 构造问题、调 Rust 迭代器、解释结果；任务级 Facade、算法链（族初猜 → Rust 修正 → 高保真传播）、子问题构造三层编排                                                                         |
| CI        | 三平台 wheel 矩阵（Windows x64、Linux x86_64、Linux aarch64）+ sdist + GitHub Release + PyPI；CSPICE 编译包经 GitHub Release 分发；manylinux 2_28 基线覆盖麒麟等国产系统 |
| 数据管理      | GitHub Release 管编译库与星历数据；Git 管轨道族种子参数；本地计算产物入轨道库 catalog（JSON 元数据 + NPZ 数组段，SQLite 仅作派生索引）                                                   |

**坐标。**坐标系被分解为 **Axes（轴向）+ Origin（原点）**：给定历元，Axes 返回旋转矩阵、Origin 给出绝对状态，二者拼成完整的数学参考框架。轴向类型覆盖 ICRS 惯性系、SPICE 高精度 ITRF93（对 `pxform/sxform` 验证至  $10^{-12}$ ）、GMAT 兼容 ITRF、IAU 2006 简化地球定向（无 SPICE 场景），以及依赖航天器瞬时状态的动态轴 VNB 与 LVLH（推力方向与轨道保持的常用系）。会合系与 J2000 之间的批量转换已下沉 Rust，含质心 → 地心平移修正。

**联合时空变换。**依托中科院 `r2s2` 库实现  $TDT+GCRS \leftrightarrow TDB+EBCRS$  的联合时空变换，含 IAU 决议要求的相对论项；e2m2e 补齐了 EBCRS 原点平移（ $\boldsymbol{x}_{ebcrs} = \boldsymbol{x}_s - \boldsymbol{x}_{emb}(t)$ ）。双向往返验证：位置差  $< 0.001\text{ mm}$ 、时间差  $< 1\text{ }\mu\text{s}$ ；相对论修正量级在 LEO 处约  $0.20\text{ m}$ 、月球距离处约  $10.1\text{ m}$ 。

**常数。**物理常量按基准集管理（DE421 / DE440 / WGS-84 多套并存、每套内部自治、按场景选用），Python 与 Rust 同源生成，杜绝两侧漂移。地月 CR3BP 基准取 DE421 自治值： $\mu = 0.01215058535$ ，特征长度  $DU = 384400\text{ km}$ ，特征时间  $TU \approx 375190\text{ s}$ （约 4.34 天）。

3.2 三层动力学模型

e2m2e 按保真度提供三层动力学，并支持三者间的状态转换（表 2）。

表 2: 三层动力学模型

| 模型       | 保真度  | 用途与性质                                       |
|----------|------|---------------------------------------------|
| CR3BP    | 概念设计 | 会合系无量纲自治系统，有 Jacobi 积分与五个平动点；周期轨道族设计的载体     |
| BCR4BP   | 中间   | 叠加太阳解析摄动（位置为时间解析函数，无需星历）；系统具时间周期性，WSB 搜索的载体 |
| 星历 $N$ 体 | 高保真  | J2000 惯性系物理单位，SPICE 星历查询 $N$ 个可配置天体；任务级精确外推 |



圆型限制性三体问题（CR3BP）在会合旋转系中是无量纲自治系统，运动方程为

$$\ddot{x} - 2\dot{y} = \frac{\partial \Omega}{\partial x}, \quad \ddot{y} + 2\dot{x} = \frac{\partial \Omega}{\partial y}, \quad \ddot{z} = \frac{\partial \Omega}{\partial z}, \quad (1)$$

伪势能与两个引力距离为

$$\Omega = \frac{1}{2}(x^2 + y^2) + \frac{1-\mu}{r_1} + \frac{\mu}{r_2}, \quad r_1 = \sqrt{(x+\mu)^2 + y^2 + z^2}, \quad r_2 = \sqrt{(x-1+\mu)^2 + y^2 + z^2}, \quad (2)$$

唯一运动积分为 Jacobi 常数  $C_J = 2\Omega - v^2$ 。

双圆限制性四体问题（BCR4BP）在 CR3BP 上叠加太阳质点摄动（直接项 + 间接项），太阳位置为时间解析函数  $\mathbf{r}_s(t) = a_s(\cos \theta, \sin \theta, 0)$ ,  $\theta = \theta_0 + \omega_s t$ , 在会合系中逆行。系统不再自治但保持时间周期性，是 WSB 弹道捕获搜索的工作介质：与星历模型对比，1 天外推偏差约  $10^3$  km，2 天内优于 CR3BP。

星历  $N$  体模型以中心体点质量加第三体摄动为基本形式：

$$\mathbf{a} = -\frac{\mu_0 \mathbf{r}}{\|\mathbf{r}\|^3} - \sum_i \mu_i \left[ \frac{\mathbf{r} - \mathbf{r}_i}{\|\mathbf{r} - \mathbf{r}_i\|^3} + \frac{\mathbf{r}_i}{\|\mathbf{r}_i\|^3} \right], \quad (3)$$

天体状态来自 SPICE 星历，非自治、无平动点与 Jacobi 积分，用于任务级精确设计。三层模型均支持 42 维增广的 STM 传播（ $\delta \mathbf{x}(t) = \Phi(t, t_0) \delta \mathbf{x}(t_0)$ ）。

### 3.3 高保真力模型

星历层之上，力模型以“注册—启停”的组合容器管理，各力分量提供解析雅可比与 Rust 序列化钩子：

- **引力**：中心体点质量；第三体直接 + 间接项；完全正规化球谐引力场（Pines 递推，支持 EGM96 / GRGM900C 与自定义系数文件，内置地球固体潮与月球  $k_2 = 0.024116$  固体潮）；
- **光压**：炮弹模型（ $P_{1AU} = 4.56 \times 10^{-6}$  N/m<sup>2</sup>，可选圆锥阴影模型，多遮挡体合成与 GMAT 对齐）与 ECOM 9 参数经验模型（DFH 兼容）；
- **大气阻力**：ITRF 中计算， $C_d$  默认 2.2；密度模型取 USSA76 分段指数大气，带 F10.7 / Ap 一阶修正；
- **推力**：连续推力（方向系可选惯性 / VNB / LVLH）与变质量连续推力（质量为第 7 状态量，7 维增广 Rust 路径）；脉冲机动经分段施加；
- **相对论修正**：后牛顿 Schwarzschild + Lense-Thirring + de Sitter 项，与 GMAT 对齐。

精度验证：满配力模型传播与 GMAT、DFH 对齐至亚百米量级；对 DFH 满配算例，Halo 7 天偏差 88 m、Lissajous 7 天偏差 201 m，残余差异的 97% 来自 ECOM 与炮弹光压模型本身的不同。按 ADR 0013 的原则，GMAT/DFH 仅作开发期交叉参考，正确性由物理定义裁决。

### 3.4 Rust 积分器内核

重迭代计算全部在 Rust 侧完成，提供三族积分器（表 3）。自适应 RK 的误差控制取  $\text{tol} \cdot \max(1, \|y\|)$  阈值，STM 增广分量不参与步长控制（与 GMAT 一致）。基准对比（归一化



LEO+J2、1 天、 $\text{tol} = 10^{-13}$ )：PD45 约 5000 步  $\times$  7 次求值，RK89 约 800 步  $\times$  16 次，ABM 约 500 步  $\times$  2 次——高阶与多步方法显著降低右端函数求值总数。

表 3: Rust 积分器三族

| 族          | 方法                 | 阶数        | 典型场景                |
|------------|--------------------|-----------|---------------------|
| RK 自适应单步   | PD45 / PD78 / RK89 | 5 / 8 / 9 | 通用传播与精度基准           |
| Adams 多步   | ABM (PECE)         | 4         | 光滑右端长弧段，每步仅 2 次右端求值 |
| Cowell 双积分 | Störmer–Cowell     | 8         | 纯引力轨道，状态维度减半        |

两个性能关键设计：其一，**星历缓存**——SPICE 查询以均匀网格预采样 + 三次样条 ( $C^2$  连续) 注入 Rust 侧，满配力模型传播加速 10 倍以上，同时解除了 CSPICE 全局状态对并行的限制；其二，**编译路径**——力模型一次序列化后整个积分循环在 Rust 内完成，无 Python 往返。事件检测遵循 scipy 语义 (`terminal / direction`)，Poincaré 截面可直接生成事件函数供积分内检测。

## 4 周期轨道设计

### 4.1 轨道族总览

地月空间的可用轨道按离月距离分为四层，尺度是主线：离月越远，地球引力占比越高，动力学从二体开普勒过渡到三体问题，轨道从稳定变为不稳定（表 4）。e2m2e 提供统一的轨道族生成入口 `orbit_family_generation()`，覆盖八类轨道族；Lyapunov 与共振轨道（RO）在微分修正策略层额外支持。

表 4: 周期轨道族一览

| 轨道族         | 位置       | 特点与用途                                             |
|-------------|----------|---------------------------------------------------|
| DRO         | 月心       | 远距逆行，中性稳定、维持成本低，月轨空间站候选                           |
| DPO         | L2 附近    | 顺行远距，不稳定，可用于月球背面通信中继                              |
| Halo / NRHO | L1/L2    | 三维周期；NRHO 为高振幅近直线形态 ( $A_z > 0.1$ 无量纲)，Gateway 采用 |
| Axial       | L1/L2    | Gómez Type B 分岔，轴向运动，覆盖特性与 Halo 互补                |
| Lyapunov    | L1/L2/L3 | 平面周期轨道                                            |
| Lissajous   | L1/L2/L3 | 拟周期（面内/面外频率不可通约），非线性中心约化流上生成                      |
| SPO / LPO   | L4/L5    | 短周期 / 长周期轨道，大振幅 LPO 呈马蹄形                          |
| Horseshoe   | L4/L5    | LPO 链的大振幅成员                                       |
| RO          | —        | 共振轨道， $y$ 轴对称出发                                   |

族的周期语义严格区分：Halo、NRHO、Axial、SPO、LPO、Horseshoe、DRO 成员严格周期；Lissajous 成员为拟周期有界轨迹，不做周期闭合。

## 4.2 初值猜测：从解析近似出发

周期轨道设计的第一块基石是初猜。e2m2e 的初猜来源有三：

- **Richardson 三阶解析近似**（1980）：共线平动点附近 Halo 轨道的三阶解析解。内部流程为：求解平动点距离参数  $\gamma$ （五次方程，Brent 法）→ 计算面内频率  $\omega_p$  → 组装三阶系数与解析解。小振幅种子（如无量纲  $z_0 = 0.001$ ）精度最高，交由延拓放大；
- **种子轨道**：DRO 从标准种子（振幅约 90786 km）出发，固定  $x$  轴穿越点做微分修正并双向延拓，包络覆盖 1737–110000 km；
- **Normal Form 表征参数**：把 CR3BP 平动点附近的动力学化简为作用量–角变量形式的表征参数  $[q_1, p_1, I_2, \theta_2, I_3, \theta_3]$ ，用几个常数描述本需六个状态量随时间演化的轨道。流水线为：星历轨道初值 → 动力学替代轨道 → quasi-Floquet 变换 → 中心流形化简 → 表征参数；其中 quasi-Floquet 化简可选 36 维矩阵法或 21 维  $\mathfrak{sp}(6)$  Lie 代数参数化（构造上保辛），中心流形 Lie 变换默认截断 10 阶。

## 4.3 微分修正：利用对称性的 STM 牛顿迭代

微分修正迭代调整初始条件使轨道满足周期性约束，核心是状态转移矩阵（STM）牛顿迭代。e2m2e 充分利用 CR3BP 的三种镜像对称性来缩小问题规模——例如 XZ 平面对称意味着：若  $(x, y, z, \dot{x}, \dot{y}, \dot{z})$  是解，则  $(x, -y, z, -\dot{x}, \dot{y}, -\dot{z})$  也是解；于是 Halo 轨道只需从 XZ 平面（ $y = 0$ ）垂直出发，修正到半周期后再次垂直穿越即可。

对称性与固定参数的不同组合构成 11 种修正策略（Lyapunov/DRO 固定  $x_0$ 、固定周期轨道固定  $T_{1/2}$ 、RO 固定  $y_0$ 、Halo 固定  $z_0$  或  $x_0$ 、Axial 固定  $v_{z0}$ 、L4/L5 的 SPO/LPO 无对称全周期闭合等）。架构上策略层与求解器严格分离：策略函数只生成不可变配置对象，执行下沉到 Rust CR3BP 修正内核。

收敛失败可诊断：历史记录给出每步误差、终止原因（正常收敛 / 修正量低于机器精度 / 发散 / 雅可比奇异 / 停滞 / 收敛到  $T \approx 0$  的寄生根），雅可比奇异提示约束线性相关、需要更换策略。

## 4.4 多重打靶：长弧段的星历修正

微分修正对初猜敏感，弧段越长越甚。多重打靶把轨道分为  $N$  段独立积分，在拼接点施加连续性约束

$$\mathbf{x}_i(t_{i+1}) = \mathbf{x}_{i+1}(t_{i+1}), \quad (4)$$

通过同时调整各段初始状态（可选含时间节点）使所有约束同时满足，把“长弧敏感”转化为“多段小修正”。标准实现的容差达  $10^{-10}$ 。

星历模型下的 patch 点修正统一走 Rust 多重打靶内核，并按轨道稳定性分派两条路径：

- **稳定轨道**（DRO）：速度加权的两级路径——修正一圈后自由外推有界；
- **不稳定轨道**（Halo/NRHO/DPO）：全程分段打靶，逐段积分填满星历网格；NRHO 每段固定 1 圈，Halo 最长 3 圈一段，DPO 每圈 64 个等时节点。

节点采样策略同样按轨道物理定制：Halo 采用近月点加密（近月窗口内 5 节点 + 窗外 8 节点），因为近月点速度大、STM 病态；NRHO 等时间采样。

#### 4.5 延拓：从一条轨道到一个族

给定一条已收敛的种子轨道，沿族参数（Jacobi 常数、振幅等）逐步微调初始条件、每步用微分修正重新收敛，即生成一族轨道。e2m2e 提供两种延拓：

- **自然参数延拓**：沿单一参数推进，每步以当前解为初值；
- **伪弧长延拓（PAL）**：增加弧长约束，可越过转向点，适合参数-振幅关系非单调的族（如 Halo 的振幅-周期曲线）。

失败处理是工程化的：某步不收敛则缩减步长重试，缩到下限仍不收敛则终止该方向，族中只追加收敛成员，诊断统计总步数 / 成功 / 失败。L2 NRHO 族采用“折叠后固定  $x_0$  延拓”的专用编排；Halo 族用固定  $z_0$  的自然参数延拓，正负  $z_0$  分北族南族。

#### 4.6 稳定性分析与不变流形

周期轨道的稳定性由单值矩阵（一个周期的 STM）决定：

$$\mathbf{M} = \Phi(T, 0). \quad (5)$$

由 Hamiltonian 系统的辛结构，特征值成对出现  $(\lambda, 1/\lambda)$ 。Broucke 稳定性指数对每对倒数特征值求和取实部：

$$\nu = \lambda + \frac{1}{\lambda}, \quad (6)$$

$|\nu| < 2$  则该模态稳定， $|\nu| > 2$  则不稳定且越大越不稳。特征值与 +1、-1 及单位圆的关系标识分岔类型（鞍结 / 倍周期 / 环面），支持族级分岔检测。数值可信度由辛结构残差自校验： $|\det \mathbf{M} - 1|$  与  $\|\mathbf{M}^\top \mathbf{J} \mathbf{M} - \mathbf{J}\|$  显著偏大时应收紧积分容差重新分析。

单值矩阵的实特征向量同时给出不变流形方向：稳定流形取  $|\lambda| < 1$  方向、不稳定流形取  $|\lambda| > 1$  方向。沿轨道相位点用 STM 转运特征向量、施加  $\pm \varepsilon$  扰动（典型 50 km/DU）得流形种子，反向 / 正向积分得到流形管，供低能量转移拼接使用（见第 5 节）。

#### 4.7 设计流水线

把上述算法串起来，一条任务轨道的完整流水线为：



数值重活（传播、打靶、批量坐标转换、批量星历查询）全部在 Rust 侧，Python 只做编排；四条典型轨道（LLO、ELFO、DRO、Halo）的端到端生成在 release 构建下总计不到 1 分钟。

### NominalOrbit: 设计与控制之间的契约

设计完成的标称轨道以 NominalOrbit 结构交付：等间隔状态表 + Floquet 基 + 投影因子。轨道保持算法直接消费该结构——设计阶段的 Floquet 分析结果不需要在控制阶段重算，两个环节通过同一份数据契约解耦。

## 5 转移轨道设计

e2m2e 的转移设计覆盖脉冲、低能量与低推力三类推进模式，核心方法论是“搜索—优化”两步法：先在粗网格上以弹道传播筛选可行解族，再把最优候选交给非线性规划（NLP）精化。统一的设计变量为切向速度比  $\alpha$ （控制出发方向）、转移时间  $T$  与远地点到插入点的时间  $t_{\text{ins}}$ ，总目标函数

$$J = \Delta v_1 + \Delta v_2, \quad (7)$$

约束包括插入点位置连续、速度方向与目标平行，以及对地球与月球的碰撞避免。动力学基准为 CR3BP，质量比  $\mu$  取自 DE421 常量基准。

### 5.1 脉冲转移

#### 5.1.1 Lambert 求解与 porkchop 扫描

Lambert 求解器采用 Izzo (2015) 算法，内核在 Rust 侧（Python 仅做类型转换与封装）：支持短路径/长路径方向与多圈解（多圈返回低能右分支），并提供批量接口—— $N$  组几何  $\times M$  个飞行时间的网格一次进入 Rust 求解，不可行组合填 NaN 而不影响其余。求解器经 Vallado 经典算例验证。

在出发时刻  $\times$  飞行时间网格上逐点解 Lambert 即得 porkchop 双脉冲  $\Delta V$  图，端点状态经终端条件抽象（TerminalCondition）提取，解析终端无需动力学传播。

#### 5.1.2 三体打靶

二体 Lambert 在地月空间只配当初猜。三体 Lambert 打靶用二体解播种 CR3BP 下的阻尼牛顿迭代：带状态转移矩阵  $\Phi(t, t_0)$  传播，每步修正量由末端 STM 的  $\Phi_{rv}$  块解出；全步使误差增大时步长减半；终端位置误差小于  $10^{-8}$ （无量纲）判收敛。

#### 5.1.3 多脉冲优化与 Lawden 主矢量检验

固定端点间的  $n$  脉冲转移：决策变量仅为中途节点的  $[t_i, \mathbf{r}_i]$ ，相邻节点间弧段由 Lambert 封闭（可切三体打靶精修），以 SLSQP 最小化总  $\Delta V$ 。双脉冲解是否还能改进，用 Lawden 主矢量检验回答：由横截条件  $\mathbf{p}(t_0) = \hat{\Delta}v_0$ 、 $\mathbf{p}(t_f) = \hat{\Delta}v_f$  出发，协态经 STM 携带得到  $\mathbf{p}(t)$  曲线，最优性必要条件为

$$\|\mathbf{p}(t)\| \leq 1 \quad \text{全程成立}, \quad \text{脉冲点 } \|\mathbf{p}\| = 1 \text{ 且与脉冲同向}. \quad (8)$$

弧内  $\|\mathbf{p}\| > 1$  表明插入中途脉冲可进一步降低总  $\Delta V$  (Lion & Handelsman, 1968), 检验同时给出建议的插入时刻与位置, 直接播种三脉冲优化。LEO→GEO 算例复现霍曼基准 3.7708 km/s。

此外, 经典 Hohmann 双脉冲最小能量转移

$$\Delta v_1 = \sqrt{\frac{\mu}{r_1}} \left( \sqrt{\frac{2r_2}{r_1 + r_2}} - 1 \right), \quad \Delta v_2 = \sqrt{\frac{\mu}{r_2}} \left( 1 - \sqrt{\frac{2r_1}{r_1 + r_2}} \right) \quad (9)$$

作为解析基准与复杂转移的初猜来源提供。

## 5.2 低能量转移

### 5.2.1 月球借力 (LGA)

采用圆锥曲线拼接的三段式设计: 双曲线脱离出发轨道 → 月球影响球内借力改变速度矢量 → 双曲线插入目标轨道。搜索接口在 CR3BP 无量纲状态下工作, 典型搜索域为飞行时间 15–45 天、总  $\Delta v$  上限 4.0 km/s, 结果报告总代价、飞行时间与近月点高度。

### 5.2.2 WSB 弹道捕获

利用日地弱稳定边界 (WSB): 航天器飞出月球影响球进入太阳摄动下的远端弧段, 远地点附近太阳引力做功, 使月心相对速度低于捕获阈值, 以近零  $\Delta v$  弹道捕获 (Belbruno & Miller, 1993)。捕获判据为近月点处二体 Kepler 能量

$$H_2 = \frac{\|\mathbf{v}\|^2}{2} - \frac{\mu_{\text{moon}}}{r} < 0. \quad (10)$$

求解分两步: **弹道网格搜索**在 BCR4BP (双圆限制性四体问题) 下前向传播, 于太阳相位  $\times$  出发相位  $\times$  飞行时间网格上接近月点高度与  $H_2$  判据筛选——传播、截面检测与筛选全部在 Rust 侧以 Rayon 并行 (Python 对照实现仅在显式指定时运行, 绝不自动回退); **到达段精化**用三体 Lambert 打靶闭环最优候选的月心到达弧。典型搜索域为飞行时间 90–150 天。

### 5.2.3 不变流形拼接

第三条低能量路线利用周期轨道自身的不变流形管: 出发轨道的不稳定流形与目标轨道的稳定流形在同一 Poincaré 截面 (平面截面或近拱点截面  $s = \mathbf{r} \cdot \mathbf{v} = 0$ ) 上的穿越点两两配对, 按加权拼接代价

$$w_r \|\Delta \mathbf{r}\| + w_v \|\Delta \mathbf{v}\| \quad (11)$$

升序筛选候选。端到端流水线串联为: 出发不稳定流形  $\times$  目标稳定流形 ( $\pm$  分支四种组合全局取最优) 传播到次天体近拱点截面取最优拼接, 拼接点之后由三体 Lambert 打靶闭合到目标轨道; 脉冲相应分为出发、拼接、到达三段。基准算例 (L1 Lyapunov 族中等到大幅值轨道) 的拼接脉冲在几十 m/s 量级。当前该路线基于 CR3BP, 星历接入已预留接口。

低能量路线的价值在于: 以时间换燃料, 为经济型着陆器/入轨器方案乃至应急低燃料救援提供通道。



### 5.3 低推力转移

低推力段采用 7 维增广状态  $[\mathbf{r}, \mathbf{v}, m]$ ，动力学为

$$\dot{\mathbf{r}} = \mathbf{v}, \quad \dot{\mathbf{v}} = \mathbf{a}_{\text{grav}} + \frac{T \delta}{m} \hat{\mathbf{u}}, \quad \dot{m} = -\frac{T \delta}{I_{sp} g_0}, \tag{12}$$

其中油门  $\delta \in [0, 1]$ ，推力方向  $\hat{\mathbf{u}}$  由球面角  $(\theta_1, \theta_2)$  参数化。求解是两级闭环：**Q-law** 初猜以 Lyapunov 反馈律前向积分产生次优控制历史；**SLSQP 打磨**在每段常值控制  $(\delta, \theta_1, \theta_2)$  上做最小燃料 NLP，终端位置速度匹配目标。两种求解器可选：打靶法（解析雅可比，快 5–24 倍）与 Hermite–Simpson 配点法（大规模问题更鲁棒）。

脉冲与低推力的代价口径经等效  $\Delta v$  区分：脉冲为  $\Delta v = \sum_i \|\Delta \mathbf{v}_i\|$ ，低推力用齐奥尔科夫斯基反演  $\Delta v_{\text{eq}} = I_{sp} g_0 \ln(m_0/m_f)$ ；由于引力损耗， $\Delta v_{\text{eq}} \geq$  脉冲  $\Delta V$ ，大能量转移上差异显著。

### 5.4 网格搜索 + NLP 两步优化框架

第一步**网格搜索**：在出发轨道上等时间采样出发点（典型 200 个），每个出发点在  $\alpha \in [0.5, 2.5]$  内均匀采样（典型 101 个），前向积分转移弧，经碰撞（地球 200 km / 月球 100 km）、相交与最小距离（ $10^{-3}$ ）约束筛选可行解；数值内核已下沉 Rust 并行执行。

第二步**NLP 精化**：决策变量  $\mathbf{y} = \{\alpha, T, t_{\text{ins}}\}$ ，目标  $J(\mathbf{y}) = \Delta v_1 + \Delta v_2$ ，约束为插入点位置连续（等式）与速度平行（等式，或松弛为夹角容差 0.05 rad 的不等式）。求解器默认 SciPy SLSQP，可选 COPT 商业优化器并在失败时回退。

表 5：转移设计能力一览

| 类别  | 方法             | 要点                                    |
|-----|----------------|---------------------------------------|
| 脉冲  | Lambert (Izzo) | Rust 内核，批量网格求解，短/长路径与多圈               |
| 脉冲  | 三体打靶           | 阻尼牛顿 + STM，位置误差 $< 10^{-8}$ 收敛        |
| 脉冲  | 多脉冲            | SLSQP + Lawden 主矢量检验，自动建议插入脉冲         |
| 脉冲  | Hohmann        | 解析基准与初猜                               |
| 低能量 | LGA            | 圆锥曲线拼接三段式，TOF 15–45 天                 |
| 低能量 | WSB            | BCR4BP + $H_2 < 0$ 判据，Rust Rayon 并行搜索 |
| 低能量 | 流形拼接           | Poincaré 截面配对 + 三体打靶闭合，拼接脉冲 m/s 级     |
| 低推力 | Q-law + NLP    | 打靶（解析雅可比 5–24×）或配点                    |
| 框架  | 搜索–优化          | 网格搜索（Rust 并行）→ SLSQP/COPT 精化          |

## 6 轨道控制与 Monte Carlo 仿真

不稳定轨道（Halo/NRHO/DPO）必须配合轨道保持才能真正可用。e2m2e 把控制仿真作为设计链的最后一环：设计阶段产出的 NominalOrbit（等间隔状态表 + Floquet 基 + 投影因子）直接被控制算法消费，Floquet 分析结果无需重算。

## 6.1 三种控制律

- **特征点控制**：只在轨道的特征位置（如近月点、平面穿越点）施加修正，修正次数少、燃耗低，适合长期保持；
- **严格目标点控制**：把状态精确打回标称轨道上的目标点，跟踪精度最高；
- **松弛目标点控制**：目标点容许误差带，在跟踪精度与推进消耗之间折中。

此外提供角动量管理功能：姿态与推力器联合控制，把动量卸载纳入保持策略。

## 6.2 Monte Carlo 误差仿真

轨道保持的燃耗统计必须在误差模型下评估。e2m2e 支持导航误差与推力误差联合注入的 Monte Carlo 仿真：对同一标称轨道批量运行保持仿真，统计  $\Delta V$  消耗分布与轨道维持情况，为任务论证给出置信区间而非单点估值。仿真产物自动归档入轨道库并支持标签管理——在 MCP 场景下，“对这条 NRHO 跑 100 次 Monte Carlo 保持仿真并打上 **candidate** 标签”是一条自然语言指令即可完成的工作流。

# 7 接口、数据与生态

## 7.1 Facade：唯一的任务级入口

所有能力收敛到 Facade 一个类（ADR 0014），CLI 与 MCP 服务由 Facade 方法元数据同源派生——任务级 API 模型除执行输入校验外，还公开当前任务上下文下的参数元数据（条件取值域与校验器共用同一份规则定义），供 GUI、CLI、MCP 生成控件与范围提示，外部消费者不维护本地参数规则副本。任务级方法分三档：

- **一档任务**：design\_orbit（任务轨道设计：CR3BP 初猜 → 星历修正 → 高精度预报）、control\_orbit（轨道保持 Monte Carlo 仿真）、transfer\_design（HMN/LGA/WSB/低推力）、orbit\_propagation（轨道预报），以及 spacetime\_transform（会合系与 J2000、GCRS 与 EBCRS 的时空互转）；
- **二档子任务**：orbit\_family\_generation（八族延拓生成，族记录自动入库）等；
- **轨道库**：7 个 catalog 方法（见下）。

**Listing 1:** 设计一条地月 L2 Halo 轨道（examples/main\_design.py）

```
1 from e2m2e.api import Facade
2
3 facade = Facade()
4
5 result = facade.design_orbit(
6     orbit_type="Halo",
7     collinear_point=2,
8     amplitude=30000.0,          # km
9     epoch=[2024, 1, 1, 0, 0, 0.0],
```



```

10     duration=365.25 * 86400.0,    # s
11 )
12
13 print(result.orbit_type)
14 print(result.initial_state)

```

## 7.2 MCP: Agent 原生接口

e2m2e 可作为 MCP 服务器 (e2m2e mcp-serve, stdio 传输, 不监听端口) 向 LLM Agent 暴露 13 个任务级工具: 6 个任务方法 (一档 5 个 + 族生成) 与 7 个轨道库工具。工具清单由 Facade 方法元数据纯派生, 无需另行维护。所有响应走统一信封 {status, data, error, meta}: 错误以错误码表达 (INVALID\_PARAMS、RECORD\_NOT\_FOUND、PROPAGATION\_FAILED 等), 不泄漏 traceback; 数值工具以 data.status 报告 converged / diverged / stagnated / max\_iterations 软失败语义 (ADR 0020) —— Agent 可以把“未收敛”当作正常信息来处理, 而不是一次异常。

配置方式是在 MCP 客户端注册一条命令; 建议把 cwd 钉在含 kernels/ (SPICE 内核) 与 catalog/ (轨道库) 的目录, 或以 SPICE\_KERNEL\_DIR / E2M2E\_CATALOG\_DIR 指定绝对路径。配置完成后即可用自然语言驱动, 例如:

设计一条近月点高度 3000 km 的 L2 南族 NRHO, 然后对它做 100 次 Monte Carlo 轨道保持仿真, 结果打上 “candidate” 标签。

## 7.3 轨道库 catalog

计算产物自动归档 (ADR 0031): 记录 = JSON 元数据 + NPZ 数组段 (带 schema 版本号), SQLite 仅作派生索引。产物型工具成功后返回 record\_id, 可跨工具链式引用 (例如 control\_orbit 以 input\_record\_id 引用 design\_orbit 的产物)。7 个库工具覆盖完整生命周期: catalog\_query (按族 / 平动点 / Jacobi 区间 / 振幅 / 标签 / 收敛状态多维过滤)、catalog\_get、catalog\_delete、catalog\_tag (教学标注)、catalog\_promote (族成员提升为独立记录)、catalog\_export (打包导出)、catalog\_sweep (参数空间批量扫描入库)。

## 7.4 安装与部署

Listing 2: 安装与 SPICE 内核配置

```

1 uv pip install e2m2e                # 发布 wheel 内嵌 CSPICE, 开箱即用
2 uv pip install "e2m2e[mcp]"         # MCP 服务
3
4 # 源码开发 (需 Rust 工具链):
5 git clone https://github.com/cislunarspace/e2m2e.git
6 cd e2m2e
7 make dev                            # 同步依赖 + 拉取 CSPICE 编译包与 SPICE 内核 + 构建 Rust 扩展

```

发布 wheel 覆盖 Windows x86\_64、Linux x86\_64 与 Linux aarch64（鲲鹏 / 飞腾 / 树莓派），内嵌静态链接的 CSPICE，Rust 快速路径（STM 传播、打靶、第三体引力等）开箱即用；扩展不可用时显式报错而非静默降级。SPICE 内核打包为 GitHub Release 资产 `kernels-v1`（`make setup` 自动下载），包含 DE 行星星历、地球自转、月球姿态、闰秒与行星常数内核；r2s2 时空转换另需含 TT-TDB 时间星历的历表。官方来源为 NASA NAIF。

`examples/` 目录提供六个可运行示例：端到端轨道设计、轨道保持 Monte Carlo、高保真传播、Lambert 转移、环月轨道四层对比（LLO/ELFO/ DRO/Halo，全量运行不到 1 分钟）、轨道族生成。

## 8 与现有工具的对比分析

### 8.1 对比范围与方法

本章将 e2m2e 置于现有航天软件生态中定位。对比对象共 10 个，覆盖四类形态：通用任务分析平台（GMAT、STK、ATK）、开源天体力学库（Orekit、pykep、poliastro）、自建路线的起点与精度标尺（SciPy）、生态互补件（r2s2、Cesium、ootk）。调研于 2026 年 8 月完成，版本锚点为 e2m2e 5.8.7：对 GMAT、Orekit、pykep、poliastro、ootk、r2s2 做了本地代码级调研（逐仓库 grep 与源码通读），对商业闭源的 STK、ATK 及纯渲染层的 Cesium 采用官方文档与第三方文献调研；文中所有数字均可在各对象的仓库或官方文档中复核。

为保证叙述公允，本章遵循三条原则：

1. **无对拍不倍数**：凡缺乏同场景对拍数据的对象（STK、Orekit、ATK），只作架构性对比，不宣称性能倍数；
2. **性能分区陈述**：对 SciPy 自建路线的加速可定量（回调开销是结构性的）；对 GMAT 的差异在并行架构层面陈述；
3. **精度谈对齐不谈胜负**：e2m2e 的力模型层以 GMAT 数学规格为移植基准，精度关系是“对齐声明 + 组件级移植一致性”，而非“谁更准”。

### 8.2 生态全景

### 8.3 核心差异化轴：地月三体设计自动化

e2m2e 与所有对比对象的分水岭在于：三体能力是“设计流水线”还是“零件或框架”（表 7）。

结论是清晰的：八个对比对象无一同时具备“八族生成 + 流形低能转移 + 三体轨道保持 + 星历修正链”。GMAT 的三体是几何点加  $N$  体求和，STK 提供框架但 workflow 靠手工搭序列，Orekit 提供构件但装配靠用户，pykep 把三体当作优化载体而非设计对象，poliastro 的三体从未进入主库。“地月三体设计自动化”因此是 e2m2e 的独特生态位。

### 8.4 代表性对象逐评

表 6: 对比对象全景（2026 年 8 月版本锚点）

| 工具        | 开发方 / 许可               | 定位（语言）                         | 与 e2m2e 的关系                        |
|-----------|------------------------|--------------------------------|------------------------------------|
| GMAT      | NASA / Apache-2.0      | 通用任务分析平台，GUI + 插件（C++，约 85 万行） | e2m2e 的力模型对齐基准；无 CR3BP 内核          |
| STK       | Ansys / 商业闭源           | 数字任务工程全功能平台                    | 行业事实标准；Astrogator 有 CR3BP 框架但无族自动化 |
| ATK       | 国防科大 / 闭源二进制           | STK 国产替代，300+ 单位               | 通用 MCS 框架；公开材料未见三体专用工具             |
| Orekit    | CS GROUP / Apache-2.0  | 飞行动力学基础设施库（Java，约 47 万行）       | 最成熟基础设施；三体仅 Halo/Lyapunov 两型构件     |
| pykep     | ESA / MPL-2.0          | 行星际轨迹优化研究库（C++23 + pybind）     | 三体仅积分器与基准题；非运行级工具                  |
| poliastro | 社区 / MIT               | 教学级天体力学（Python + numba）        | 已归档（2023-10）；三体停在 contrib 草稿       |
| SciPy     | 社区 / BSD 系             | 通用科学计算（Python + C/Fortran）     | 自建路线的起点，同时是 e2m2e 的精度标尺            |
| ootk      | thkruz / AGPL-3.0      | Web 端地球轨道 SSA 工具库（TypeScript）  | 赛道不同，几乎零重叠，互补                      |
| r2s2      | 紫金山天文台 / GPL-3.0       | 相对论时空坐标转换（Python，约 1.2 千行）     | e2m2e 的底层依赖，非竞品                    |
| Cesium    | Cesium GS / Apache-2.0 | 三维可视化引擎（JS/TS, WebGL）          | 纯渲染层零计算，天然前后端组合                    |

表 7: 地月三体能力对比（白皮书核心差异化轴）

| 工具     | CR3BP 动力学 / 族生成                                         | 低能转移 / 保持 / 星历修正                                                                      |
|--------|---------------------------------------------------------|---------------------------------------------------------------------------------------|
| e2m2e  | 原生三模型（CR3BP/BCR4BP/星历 $N$ 体）互转；八族自动生成 + 延拓 + 能量窗口批量     | 流形拼接 + LGA/WSB；三控制律 + Monte Carlo；初猜 $\rightarrow$ 星历修正（20 m 收敛） $\rightarrow$ 高保真三段链 |
| GMAT   | 无 CR3BP 模块（全仓 grep 零命中）；平动点仅为几何点，旋转系靠用户自拼               | 均无；星历传播为回放器形态                                                                         |
| STK    | Astrogator 11.7 起内建 CR3BP 力模型 + 差分打靶                    | 无模板化族生成（未见公开证据）；低能转移靠 MCS 自建；无保持控制律                                                   |
| ATK    | 高精度 $N$ 体数值传播 + 通用 MCS（11 段型 $\times$ 60 约束）            | 公开材料未宣称族生成/流形；有复现 GMAT 算例的 Halo 维持论文证据                                                |
| Orekit | CR3BPForceModel + STM，构件式                               | 仅 Halo/Lyapunov 两型；流形仅初值单点；无保持；打靶可配星历（用户装配）                                           |
| pykep  | heyoka 符号工厂 + 42 维变分方程                                  | 无族生成（grep 零命中）；无流形；14 个地月低推力基准题——其输入恰是 e2m2e 的产出物                                     |
| 其余四者   | SciPy 零轨道知识；poliastro 三体停在 contrib 草稿；ootk/Cesium 无三体代码 | —                                                                                     |

### 8.4.1 通用任务分析平台：GMAT、STK、ATK

**GMAT** (NASA, 约 85 万行 C++) 是与 e2m2e 关系最“近”的对象：e2m2e 的 PD78/RK89 权重、锥形阴影、相对论三项、STM 步进误差控制语义、EOP/闰秒 fixtures 均以 GMAT R2026a 数学规格为移植基准，力模型层可以说是“GMAT 数学规格的 Rust 重实现 + 扩展”。反过来，GMAT 的求解器与优化生态 (DifferentialCorrector、Yukon SQP、CSALT 直接配点、Bulirsch-Stoer 积分器) 与文档级 V&V 体系 (数学规格书、飞行资格文档) 是 e2m2e 尚未有的厚度；其无并行传播内核 (物理循环 grep 零命中) 则是 2000 年代桌面软件形态与 e2m2e 服务化形态的代际差异。二者是“对标-差异化”关系：GMAT 出校核与运营分析，e2m2e 出三体设计流水线。

**STK** (Ansys, 商业闭源) 是行业事实标准，Astrogator 自 11.7 起内建 CR3BP 力模型与差分打靶，Artemis II 任务的 Orion 跟踪可视化即由 STK 驱动。其与 e2m2e 的对比可归为四轴：许可模式 (闭源商业 vs Apache-2.0)、三体自动化 (框架 vs 一键族生成)、成本 (政府采购记录中单 Pro token 订单 11.45 万美元 vs 免费)、可编程性 (Integration 许可模块 vs 原生 MCP)。STK 的覆盖/链路/调度/定轨 (ODTK) 模块生态 e2m2e 均不覆盖，两者在典型工作流程中是“开源计算 + 商业呈现”的分层组合。

**ATK** (国防科大, 闭源二进制) 走“中国版 STK”路线：积木式机动规划框架 (11 种轨道段类型、19 种停止条件、11 类 60 种约束组件，统一 NLP + 微分修正/SQP/智能优化三类求解器)，有天宫、神舟、天舟等载人航天实战背书。其论文算例复现了地月自由返回与 GMAT 内置的 L2 Halo 维持案例 (四次机动  $-0.25/1.64/8.05/6.44$  m/s)，说明框架可承载三体问题；但公开材料中未见 CR3BP 传播器、平动点轨道族生成或不变流形自动化——这仍是 e2m2e 最干净的差异点。“平台 vs 库”、“闭源 vs 开源”、“通用 MCS vs 三体专用流水线”是两者对比的三条主线。

### 8.4.2 开源天体力学库：Orekit、pykep、poliastro

**Orekit** (CS GROUP, 约 47 万行 Java, 2002 年起 24 年积累) 是工程级基础设施的标杆：帧/时间链对官方 SOFA 库逐元素对照至  $10^{-12}$ ，传播器谱系覆盖数值/解析/DSST 半解析/SGP4/GNSS，定轨全家桶 (批最小二乘 + EKF/UKF)、80 余种事件检测器都是 e2m2e 没有的深度。但其三体能力是构件式的——动力学模型 + Richardson 初猜 + 打靶零件齐备，轨道类型仅 Halo/Lyapunov 两枚举，无族延拓、无 Lissajous/DRO、无流形批量积分与拼接、无保持控制律。两者同为 Apache-2.0，“Orekit 管地球段与定轨、e2m2e 管地月段设计”是自然的分工。

**pykep** (ESA, kep3, MPL-2.0) 是行星际轨迹优化研究库：heyoka LLVM JIT Taylor 积分器 (公差至  $10^{-16}$ ) + pagmo 进化优化, MGA 编码、Sims-Flanagan/ZOH 低推力转录、Pontryagin 间接法，官方自述“非运行级飞行动力学工具”。其三体能力止于符号动力学工厂与 14 个地月低推力基准题 (TOPS: Halo-to-Halo、DRO-to-NRHO 等)——未实现平动点计算、族构造、单值矩阵与流形打靶；而 TOPS 基准题的输入 (Halo/DRO/NRHO 标称轨道) 恰是 e2m2e 的输出。“e2m2e 生成标称轨道  $\rightarrow$  pykep + pygmo 做低推力全局优化  $\rightarrow$  e2m2e 高保真验证与保

持”是一条现成的分工链。

**poliastro** (MIT, 2023 年 10 月归档) 对本白皮书的意义在于 **生态空位的证据**: 它的单位安全 API 与 plotly 可视化是教学级精品, 但地月三体能力停留在 contrib 草稿目录 (其 README 自列待办: 微分修正、族延拓、Halo 三阶解析解、单值矩阵、流形均未实现), 力模型仅 6 个摄动函数、测试容差  $10^{-4} \sim 10^{-2}$  级。Python 生态中最活跃的天体力学教学库归档且未竟三体之作, 正是 e2m2e 所填的空。

### 8.4.3 自建路线与精度标尺: SciPy

SciPy 对轨道一无所知——无时间系统、无参考架、无星历、无力模型, 但 `solve_ivp + root` 搭 CR3BP 打靶是学术界自建的事实标准路线, 也是 e2m2e 用户会先尝试的“下位替代”。性能上其显式 RK 族为 Python 实现, 每步回调右端函数, 紧容差大批量打靶场景比编译内核慢 1–3 个数量级 (社区共识; e2m2e 侧对应证据: 同一线程化问题 Python 多进程仅得 Rust 下沉后加速的约 1/10)。精度上两者打平——e2m2e 的测试体系恰恰以 scipy DOP853 ( $10^{-13}$ ) 为金标准参考解, LEO+J2 一天弧一致性  $< 10^{-9}$ 。差距不在精度, 而在“要达到同样精度, 用户需自建整个领域层”。

### 8.4.4 生态互补件: r2s2、Cesium、ootk

**r2s2** (紫金山天文台, 约 1150 行纯 Python) 不是竞品而是 e2m2e 的底层依赖: GCRS 与 EBCRS 之间的相对论时空转换即由它承载, 反向转换默认收敛  $1 \text{ ns} / 1 \text{ mm}$ 。e2m2e 对它的独立验证 (往返一致性、与 SPICE 链路差分、与 ERFA `dtdb` 差分) 反而构成其最主要的测试覆盖。许可上 r2s2 为 GPL-3.0, 闭源集成方需注意传染性。

**Cesium** (CesiumJS, Apache-2.0) 是纯渲染层, 零轨道力学——卫星位置必须外部喂入, 历史上本就“为跟踪空间物体而生”, 对地月大尺度场景有一等公民支持 (ICRF 惯性系渲染、Artemis II 环月跟踪实证)。e2m2e 的可视化恰是短板, “EphemerisTable  $\rightarrow$  CZML  $\rightarrow$  Cesium”是零许可冲突的组合路径 (Orekit + OreCZML 已验证同一模板); 需牢记的分工是: Cesium 忠实渲染喂给它的轨迹, 轨迹本身的正确性由 e2m2e 负责。

**ootk** (TypeScript, AGPL-3.0) 在另一条赛道: SGP4/TLE 编目、传感器、碰撞、Web 端 SSA, 与 e2m2e 几乎零重叠。其价值有二: 一是互补 (编目/观测端归 ootk, 高保真任务设计归 e2m2e); 二是方法论参照——ootk 以 AFSPC 官方 SGP4Prop 位级快照与 Vallado 验证向量做回归、以 SLR 精密星历做独立真值, 这种“对外部权威实现的位级对齐”正是 e2m2e 验证体系升级可借鉴的方向。

## 8.5 性能与精度: 证据的分区陈述

e2m2e 自身的性能数字均可复现: 转移网格搜索 Rust 化加速 5.5–10.4 $\times$ ; 180 天 Halo 端到端设计 223 s  $\rightarrow$  63 s; SRP 走 Rust STM 快速路径后打靶提速 9.4 $\times$ ; 低推力解析雅可比 3–24 $\times$ 。按本章开头的原则, 这些数字只与 SciPy 自建路线对量级、与 GMAT 谈架构, 与 STK/Orekit/ATK 不作相互倍数声明——三者均无同场景公开对拍数据。



表 8: 并行/加速架构与公开量化数字

| 工具     | 架构                            | 公开数字                        |
|--------|-------------------------------|-----------------------------|
| e2m2e  | Rust 内核 + Rayon（打靶/网格/流形/WSB） | 可复现，benchmark 脚本在库          |
| GMAT   | 无并行传播内核（单实例顺序仿真）              | 无                           |
| STK    | Parallel Computing 多核分发       | 8 核 180 样本 70 s（仅此）         |
| Orekit | PropagatorsParallelizer       | 无公开数字                       |
| pykep  | heyoka LLVM JIT + 按容差缓存       | SF 求值 5.25 $\mu$ s；“千条轨迹/秒” |
| SciPy  | 单线程，Python 右端回调               | 比编译内核慢 1-3 个量级              |
| ootk   | 纯 JS 单线程（轻计算 $\times$ 海量调用）   | SGP4 391 ns/次               |

精度方面（表 9），各家的锚点与验证哲学不同：Orekit 与 ootk 的“对权威参考逐元素/位级对照”证据链最完整；pykep 在无量纲归一化单位下达到  $10^{-13}$ ；e2m2e 的积分器对解析解  $10^{-9} \sim 10^{-11}$ 、坐标链  $< 10^{-12}$ 、力模型对齐 DFH 至亚百米（Halo 7 天 88 m）——但按 ADR 0013 的“按定义验证”原则，外部对拍已撤出仓库，GMAT 侧仅保留可自跑的对拍脚本。精度不是各家分化点，可解问题域才是。

表 9: 精度验证锚点

| 工具     | 外部真值锚点                                   | 代表性数字                                                        |
|--------|------------------------------------------|--------------------------------------------------------------|
| e2m2e  | DFH / GMAT fixtures / scipy DOP853 / 解析解 | Halo 7 天 88 m；积分器 $10^{-9} \sim 10^{-11}$ ；ITRF $< 10^{-12}$ |
| GMAT   | V&V 文档体系 + 机构背书                          | 默认步容差 $10^{-11}$                                             |
| STK    | 第三方论文 / 社区交叉                             | RKF78 步容差 $10^{-13}$ 可配                                      |
| ATK    | 论文算例（含 GMAT Halo 算例复现）                   | 交会 100 m 瞄准达 100.23 m                                        |
| Orekit | 官方 SOFA 库逐元素对照                           | 帧变换 $10^{-12}$                                               |
| pykep  | 硬编码高精度真值                                 | CR3BP $\leq 10^{-13}$ （容差 $10^{-16}$ ）                       |
| ootk   | AFSPC 位级快照 + Vallado 向量 + SLR 星历         | SGP4 $5 \times 10^{-9}$ km                                   |
| r2s2   | （由依赖方 e2m2e 补测）                          | 迭代收敛 1 ns / 1 mm                                             |

8.6 组合生态：地月设计引擎

把视角从竞争切换到组合，e2m2e 在生态图中的位置是**地月设计引擎**：上游有时空与星历供给（NASA SPICE/DE 历表、r2s2 相对论时空转换），下游有现成的开源消费件（图 2）——Cesium 渲染、GMAT 校核与运营分析、Orekit 承接地球段与定轨、pykep 接手低推力全局优化（其地月基准题的输入正是 e2m2e 的产出）。每一条组合链的许可均可共存（r2s2 的 GPL-3.0 需闭源集成方注意）。



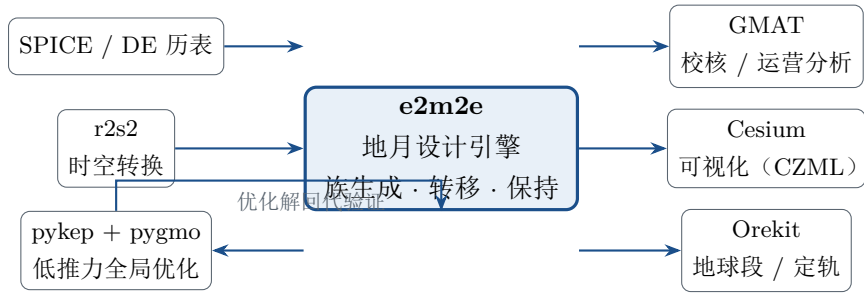


图 2: e2m2e 的组合生态：上游时空/星历供给，下游可视化、校核、定轨与全局优化各有现成开源件，e2m2e 专守地月设计引擎一层。

#### 小结：一个被反复验证的空位

通用平台没有三体内核（GMAT）、有框架没有自动化（STK/Astrogator）、有构件没有流水线（Orekit）、有动力学没有设计工具（pykep）、教学库归档在 contrib 草稿（poliastro）——八个对象无一同时具备“八族生成 + 流形低能转移 + 三体保持 + 星历修正链”。这条被代码级调研逐一证实的空位，就是 e2m2e 的生态位。

## 9 验证体系与结语

### 9.1 按定义验证

e2m2e 的验证哲学是“按定义验证”（ADR 0013）：断言主要落在解析解、守恒量与文献公式上，而不是“与自身上一次输出一致”的回归对比。测试套件按验证目标分为六类，目录与源码镜像对应：

- **algorithm**: 坐标、动力学、设计、修正、转移的算法编排链；
- **api**: Facade 与 MCP 接口的响应与错误翻译；
- **data**: 数据层——内核、参考系、物理常数；
- **mbse**: MBSE 数据模型、需求注册与图生成；
- **numerical**: 积分器精度对解析轨道、力模型加速度与雅可比；
- **tools**: 日志、格式化、可视化等辅助工具。

坐标与时间链额外与 GMAT 参考数据交叉比对；Rust 侧数值方法在 `crates/*/tests/` 中对解析解验证。高保真传播的精度对齐 GMAT 与 DFH（东方红系列平台工具）至亚百米量级。Lambert 求解器经 Vallado 经典算例验证；多脉冲优化复现 LEO→GEO 霍曼基准 3.7708 km/s；微分修正、延拓与稳定性分析经 Broucke、Richardson、Gómez 等文献公式校核。

### 9.2 结语与展望

e2m2e 把地月空间任务轨道设计的完整链条——时空基准、多保真度动力学、周期轨道族生成、三类转移设计、轨道保持仿真——收敛到一个任务级 Facade 之下，用 Rust 数值内核保证重迭代的性能与并行安全，用 MCP 接口把整条链条开放给 LLM Agent。它不做通用工具的全面覆

盖，而是在“地月空间”这个足够深的领域里，把从初猜到验证的每一步做成可组合、可追溯、可审计的工程组件。

路线图上的方向包括：星历序列化的统一中间格式 EphemerisTable 与 CCSDS OEM/ODM、SPICE BSP/PCK 等格式的互转；剩余 Python 数值（如 NLP 优化）向 Rust 内核的进一步迁移；以及以轨道库为数据基础、面向智能博弈研究的扩展。

---

|      |                                                                                             |
|------|---------------------------------------------------------------------------------------------|
| 项目地址 | <a href="https://github.com/cislunarspace/e2m2e">https://github.com/cislunarspace/e2m2e</a> |
| 在线文档 | <a href="https://cislunarspace.github.io/e2m2e/">https://cislunarspace.github.io/e2m2e/</a> |
| 许可证  | Apache 2.0                                                                                  |
| 安装   | <code>uv pip install e2m2e</code>                                                           |

---